

Caso Práctico: Registrar Ejecuciones y Elegir el Mejor Run

Adaptado para GitHub Codespaces

NIVEL
Intermedio

PROYECTO
Picos Intensidad

ENTORNO
GitHub
Codespaces

DURACIÓN
90 - 120 min

Todo el laboratorio se ejecuta desde la **terminal integrada** del Codespace (`Ctrl+``). El experimento MLflow y sus runs persisten mientras el Codespace esté activo.

1. ¿Qué estamos haciendo y por qué?

En los temas anteriores entrenamos un clasificador para detectar **picos de intensidad** usando estrategias de balanceo (Under, Over, SMOTE). El modelo base dio buenos resultados, pero nunca buscamos los mejores hiperparámetros de forma sistemática: usamos los valores por defecto.

En este tema aplicamos **Hyperparameter Tuning**: buscamos la mejor combinación con `GridSearchCV` y `RandomizedSearchCV`, registramos cada experimento en **MLflow**, comparamos los runs de forma programática y elegimos el mejor modelo para promoverlo en el Model Registry.

■ **El problema:** sin un registro estructurado de experimentos, cada prueba de hiperparámetros se pierde. No hay forma de comparar objetivamente dos modelos, reproducir el ganador ni auditar con qué datos y parámetros se entrenó. MLflow resuelve esto: cada run guarda parámetros, métricas y el modelo como artefacto.

2. ¿Qué lograrás al final?

- Entender qué es el hyperparameter tuning y por qué es crítico en MLOps.
- Aplicar **GridSearchCV** (búsqueda exhaustiva) con Stratified K-Fold.
- Aplicar **RandomizedSearchCV** (búsqueda aleatoria eficiente).
- Registrar cada experimento en MLflow con parámetros y métricas comparables.
- Comparar todos los runs programáticamente con `mlflow.search_runs()`.
- Seleccionar el mejor modelo con criterios técnicos y de negocio.

- Registrar el modelo ganador en el **MLflow Model Registry** (Staging).

3. ■ Adaptaciones para GitHub Codespaces

Aspecto	Original (Colab)	GitHub Codespaces
Entorno de ejecución	Notebook <code>.ipynb</code>	Scripts <code>.py</code> desde la terminal integrada (<code>Ctrl+`</code>)
MLflow UI	<code>localhost:5000</code> en Colab	Puerto reenviado automáticamente — pestaña PORTS
Dataset	Upload manual a Colab	Ya en el repositorio, en <code>data/</code>
<code>mlruns/</code>	Carpeta efímera de Colab	Dentro del workspace — persiste con el Codespace
Instalación de librerías	<code>!pip install</code> en celda	<code>pip install -r requirements.txt</code> en terminal

■ **Prerrequisito:** el dataset `Clasificacion_picos_intensidad.csv` debe estar en `data/` . Si no existe, el script `01_base_pipeline.py` genera uno sintético (ver Observación técnica del Paso 2).

4. Conceptos clave y estructura del proyecto

Parámetros vs. Hiperparámetros

Concepto	¿Quién lo aprende?	Ejemplo en DecisionTree	Ejemplo en SMOTE
Parámetro del modelo	El algoritmo (automático)	Umbral interno del árbol	Vectores sintéticos generados
Hiperparámetro	Nosotros (manual/ búsqueda)	<code>max_depth</code> , <code>min_samples_split</code>	<code>k_neighbors</code> , <code>sampling_strategy</code>

GridSearchCV vs. RandomizedSearchCV

Método	Combinaciones	Distribuciones	¿Garantiza el óptimo?	Uso recomendado
GridSearchCV	TODAS (exhaustivo)	No (solo listas)	Sí (en el espacio dado)	Espacio pequeño, prototipo
RandomizedSearchCV	<code>n_iter</code> (controlable)	Sí (<code>scipy.stats</code>)	No (aproximado)	Espacio grande, línea base
Bayesiano (Optuna)	<code>n_iter</code> (inteligente)	Sí	No (aproximado, mejor CV)	Producción, espacio complejo

Regla de oro — runs comparables: para comparar runs en MLflow, todos deben usar la **misma semilla** (`random_state=123`), el **mismo split** 70/30, la **misma métrica objetivo** (`f1`) y el **mismo conjunto de test**. Si cambia alguno, los runs dejan de ser comparables.

Estructura real del proyecto en el Codespace

```

Unidad06/
├── data/
│   └── Clasificacion_picos_intensidad.csv  <- dataset del proyecto (esquema minero)
├── pipeline/
│   ├── 01_base_pipeline.py                <- Secciones 1-7: base + SMOTE (línea base)
│   ├── 02_gridsearch_mlflow.py           <- Sección 8: GridSearchCV + MLflow (Run 1)
│   ├── 03_randomsearch_mlflow.py         <- Sección 9: RandomizedSearchCV + MLflow (Run 2)
│   ├── 04_comparar_runs.py                <- Sección 10: comparar runs + gráfico
│   ├── 05_mejor_modelo.py                 <- Sección 11: selección programática
│   ├── 06_model_registry.py               <- Sección 12: registro Staging → Production
│   └── run_pipeline.py                    <- orquestador: ejecuta los 6 scripts en orden
├── src/                                  <- salidas generadas (base_data.pkl, run_ids.txt, ...)
├── mlruns/                               <- tracking MLflow (regenerable, no en Git)
└── requirements.txt

```

Observación técnica — carpeta `pipeline/`, no `tema6/` : en el repositorio los scripts viven en `pipeline/` y escriben sus salidas en `src/` (por eso `src/` contiene `base_data.pkl`, `run_ids.txt`, etc.). Los comandos correctos usan la ruta `pipeline/...` (ver Paso 8).

PASO 1

Dependencias del tema

requirements.txt

Instala las librerías del tema desde la terminal del Codespace:

```
requirements.txt — requirements.txt
```

```

# requirements_tema6.txt — Dependencias del Tema 6
# Instalar en el Codespace: pip install -r requirements_tema6.txt

# Base (si no están del Tema 3)
pandas>=1.5
numpy>=1.23
scikit-learn>=1.2
imbalanced-learn>=0.10
matplotlib>=3.6
seaborn>=0.12
mlflow>=2.0
scipy>=1.9

```

```
bash — Terminal del Codespace
```

```

cd /workspaces/tu-repo
pip install -r requirements.txt

# Verificar MLflow disponible
mlflow --version

```

PASO
2

Pipeline base (Secciones 1-7)

pipeline/01_base_pipeline.py

Este script reproduce las secciones 1-7 del notebook original: carga (o genera) el dataset, hace el split 70/30, aplica las cuatro estrategias de balanceo y entrena un `DecisionTree` sin tuning. Su resultado con SMOTE es la **línea base** contra la que se compara el tuning. Exporta `src/base_data.pkl` para reutilizar exactamente el mismo split.

```
python - pipeline/01_base_pipeline.py

"""
Secciones 1-7 - Pipeline base sin tuning.

Carga (o genera) el dataset, hace el split 70/30, aplica SMOTE y
entrena un DecisionTree con parámetros por defecto.
Exporta base_data.pkl para que los scripts de tuning lo reutilicen.

Ejecutar: python src/01_base_pipeline.py
"""

import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    classification_report, accuracy_score,
    f1_score, recall_score, precision_score,
)
from imblearn.over_sampling import SMOTE

# — Sección 2: Cargar o generar dataset —————
DATA_PATH = Path("data/Clasificacion_picos_intensidad.csv")

if DATA_PATH.exists():
    df = pd.read_csv(DATA_PATH)
    print(f"Dataset cargado: {df.shape}")
else:
    # Generar dataset sintético si no existe
    print("Dataset no encontrado - generando sintético...")
    np.random.seed(123)
    n = 5000
    df = pd.DataFrame({
        "intensidad": np.random.normal(50, 15, n),
        "duracion": np.random.exponential(10, n),
        "frecuencia": np.random.uniform(1, 100, n),
        "amplitud": np.random.normal(20, 5, n),
        "temperatura": np.random.normal(25, 8, n),
        "presion": np.random.normal(1013, 20, n),
    })
    prob = 1 / (1 + np.exp(-(df["intensidad"] - 60) / 10))
    df["pico"] = (np.random.uniform(0, 1, n) < prob * 0.3).astype(int)
    DATA_PATH.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(DATA_PATH, index=False)
    print(f"Dataset sintético generado: {df.shape}")

# — Sección 3: Exploración —————
print(f"\nShape      : {df.shape}")
print(f"Columnas    : {list(df.columns)}")
```

```

print("\nDistribución del target:")
print(df["pico"].value_counts())
print(df["pico"].value_counts(normalize=True).round(4))
print("\nEstadísticas descriptivas:")
print(df.describe().round(2))

# — Sección 4: Split 70/30 —————
TARGET = "pico"
X = df.drop(columns=[TARGET])
y = df[TARGET]

# Asegurar que el target sea entero (puede llegar como float64 desde CSV)
y = y.astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=123, stratify=y
)
print(f"\nTrain: {X_train.shape} | Test: {X_test.shape}")
print(f"Train target dist: {y_train.value_counts().to_dict()}")
print(f"Test target dist: {y_test.value_counts().to_dict()}")

# — Sección 5: UnderSampling y OverSampling manuales —————
# cast a int explícito para compatibilidad con numpy.random.choice
n_minority = int(y_train.sum())
n_majority = int((y_train == 0).sum())
idx_majority = y_train[y_train == 0].index
idx_minority = y_train[y_train == 1].index

np.random.seed(123)

# UnderSampling – reducir la clase mayoritaria al tamaño de la minoritaria
idx_under = np.random.choice(idx_majority, size=n_minority, replace=False)
X_under = pd.concat([X_train.loc[idx_under], X_train.loc[idx_minority]])
y_under = pd.concat([y_train.loc[idx_under], y_train.loc[idx_minority]])

# OverSampling – duplicar la clase minoritaria al tamaño de la mayoritaria
idx_over = np.random.choice(idx_minority, size=n_majority, replace=True)
X_over = pd.concat([X_train, X_train.loc[idx_over]])
y_over = pd.concat([y_train, y_train.loc[idx_over]])

print(f"\nUnderSampling: {X_under.shape} | dist: {y_under.value_counts().to_dict()}")
print(f"OverSampling : {X_over.shape} | dist: {y_over.value_counts().to_dict()}")

# — Sección 6: SMOTE —————
smote = SMOTE(k_neighbors=5, random_state=123)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
print(f"\nPost-SMOTE: {X_train_smote.shape}")
print(f"Distribución: {pd.Series(y_train_smote).value_counts().to_dict()}")

# — Sección 7: DecisionTree base (sin tuning) – LÍNEA BASE —————
print("\n + =" * 50)
print("SECCIÓN 7 – DecisionTree base (sin tuning)")
print("=" * 50)

resultados_base = {}

for nombre, X_bal, y_bal in [
    ("Originales", X_train, y_train),
    ("UnderSampling", X_under, y_under),
    ("OverSampling", X_over, y_over),
    ("SMOTE", X_train_smote, y_train_smote),
]:
    dt = DecisionTreeClassifier(random_state=123)
    dt.fit(X_bal, y_bal)
    y_pred = dt.predict(X_test)
    resultados_base[nombre] = {

```

```

    "accuracy": round(accuracy_score(y_test, y_pred), 4),
    "f1": round(f1_score(y_test, y_pred), 4),
    "recall": round(recall_score(y_test, y_pred), 4),
    "precision": round(precision_score(y_test, y_pred, zero_division=0), 4),
}
print(f"\n[{nombre}]\n")
print(f" Accuracy : {resultados_base[nombre]['accuracy']:.4f}")
print(f" F1       : {resultados_base[nombre]['f1']:.4f}")
print(f" Recall   : {resultados_base[nombre]['recall']:.4f}")
print(f" Precision: {resultados_base[nombre]['precision']:.4f}")

print("\n=== REPORTE COMPLETO – SMOTE (línea base para tuning) ===")
dt_smote = DecisionTreeClassifier(random_state=123)
dt_smote.fit(X_train_smote, y_train_smote)
y_pred_smote = dt_smote.predict(X_test)
print(classification_report(y_test, y_pred_smote))

# — Exportar datos para scripts de tuning —————
Path("src").mkdir(exist_ok=True)
with open("src/base_data.pkl", "wb") as f:
    pickle.dump((X_train_smote, y_train_smote, X_test, y_test), f)

print("✓ Datos exportados a src/base_data.pkl")
print(" Siguiente paso: python src/02_gridsearch_mlflow.py")

```

Observación técnica — el dataset real no es el sintético: el CSV incluido tiene columnas del proceso minero (Presion, Tonelaje, Velocidad, %Solidos, Potencia, F80, Brazo, pico), no las variables sintéticas intensidad, duracion, frecuencia... que crea la rama else del script. Como el archivo **sí existe** en data/ , esa rama de generación sintética nunca se ejecuta: es código muerto con este repositorio. El código se conserva literal; solo se aclara el comportamiento real.

Observación técnica — columna índice fantasma: el CSV se guardó con su índice y encabezado vacío en la primera columna, por lo que pandas la carga como Unnamed: 0 . Como el split solo elimina pico (X = df.drop(columns=[TARGET])), esa columna entra como una feature más — es el orden de fila filtrándose al modelo. No rompe la ejecución, pero conviene corregirlo en el paso a paso del alumno:

python – Corrección sugerida (no modifica el original)

```

# Al cargar, descartar la columna índice guardada en el CSV:
df = pd.read_csv(DATA_PATH, index_col=0)

```

bash – Ejecutar

```

python pipeline/01_base_pipeline.py
# genera src/base_data.pkl con el split reutilizable

```

PASO 3

GridSearchCV + MLflow (Run 1)

pipeline/02_gridsearch_mlflow.py

Búsqueda exhaustiva sobre 120 combinaciones × 5 folds = 600 entrenamientos. El mejor estimador se evalúa en test y se registra en MLflow como *Run 1* con sus parámetros, métricas y el modelo como artefacto.

```
python - pipeline/02_gridsearch_mlflow.py

"""
Sección 8 – GridSearchCV + MLflow (Run 1).

Búsqueda exhaustiva sobre espacio de hiperparámetros definido.
120 combinaciones x 5 folds = 600 entrenamientos.
Registra el mejor en MLflow como Run 1.

Ejecutar: python src/02_gridsearch_mlflow.py
Prerrequisito: src/base_data.pkl (generado por 01_base_pipeline.py)
"""

import pickle
import mlflow
import mlflow.sklearn
from pathlib import Path

from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, f1_score, recall_score,
    precision_score, classification_report,
)

import os
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"

# — Cargar datos de la sección base —————
BASE_PKL = Path("src/base_data.pkl")
if not BASE_PKL.exists():
    raise FileNotFoundError(
        "No se encontró src/base_data.pkl. "
        "Ejecuta primero: python src/01_base_pipeline.py"
    )

with open(BASE_PKL, "rb") as f:
    X_train_smote, y_train_smote, X_test, y_test = pickle.load(f)

print(f"Datos cargados – Train: {X_train_smote.shape} | Test: {X_test.shape}")

# — 8.1 Configurar MLflow —————
# Usar ruta absoluta correcta – evita el bug de PermissionError en Codespaces
MLFLOW_DIR = Path("mlruns").resolve() # ruta absoluta real
TRACKING_URI = MLFLOW_DIR.as_uri() # file:///workspaces/.../mlruns
EXPERIMENT = "picos-intensidad-tuning"

MLFLOW_DIR.mkdir(parents=True, exist_ok=True) # crear si no existe

mlflow.set_tracking_uri(TRACKING_URI)
mlflow.set_experiment(EXPERIMENT)
print(f"MLflow tracking URI : {TRACKING_URI}")
print(f"MLflow experiment : {EXPERIMENT}")

# — 8.2 Definir espacio de búsqueda y ejecutar GridSearchCV —————
param_grid = {
    "max_depth": [3, 5, 8, 10, None], # 5 opciones
    "min_samples_split": [2, 5, 10, 20], # 4 opciones
    "min_samples_leaf": [1, 2, 4], # 3 opciones
    "criterion": ["gini", "entropy"], # 2 opciones
}

# 5 x 4 x 3 x 2 = 120 combinaciones x 5 folds = 600 entrenamientos

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=123)

gs = GridSearchCV(
```

```

    estimator      = DecisionTreeClassifier(random_state=123),
    param_grid     = param_grid,
    cv             = skf,
    scoring        = "f1",
    n_jobs         = -1,
    verbose        = 1,
    return_train_score = True,
)

print("\nEjecutando GridSearchCV (120 combinaciones x 5 folds)...")
gs.fit(X_train_smote, y_train_smote)

print(f"\nMejores parámetros : {gs.best_params_}")
print(f"Mejor F1 CV          : {gs.best_score_:.4f}")

# — 8.3 Registrar en MLflow —————
n_combinaciones = len(gs.cv_results_["params"])

with mlflow.start_run(run_name="GridSearch-DecisionTree") as run:
    # Parámetros del mejor estimador
    mlflow.log_params(gs.best_params_)
    mlflow.log_param("metodo_busqueda", "GridSearchCV")
    mlflow.log_param("n_combinaciones", n_combinaciones)
    mlflow.log_param("tecnica_balanceo", "SMOTE")
    mlflow.log_param("n_folds", 5)
    mlflow.log_param("random_state", 123)

    # Evaluar en test set
    y_pred_gs = gs.best_estimator_.predict(X_test)
    mlflow.log_metrics({
        "accuracy": round(accuracy_score(y_test, y_pred_gs), 4),
        "f1": round(f1_score(y_test, y_pred_gs), 4),
        "recall": round(recall_score(y_test, y_pred_gs), 4),
        "precision": round(precision_score(y_test, y_pred_gs, zero_division=0), 4),
        "cv_f1_mean": round(gs.best_score_, 4),
    })

    # Registrar modelo – usar artifact_path en lugar de name (compatible MLflow 2.x y 3.x)
    mlflow.sklearn.log_model(
        sk_model = gs.best_estimator_,
        artifact_path = "modelo_gridsearch",
    )
    run_id_gs = run.info.run_id
    print(f"\nRun ID GridSearch: {run_id_gs}")

print("\n=== RESULTADO GRIDSEARCH ===")
print(classification_report(y_test, y_pred_gs))

# Guardar run_id para scripts posteriores
Path("src").mkdir(exist_ok=True)
with open("src/run_ids.txt", "w") as f:
    f.write(f"gridsearch:{run_id_gs}\n")

print("✓ Run ID guardado en src/run_ids.txt")
print(" Siguiente paso: python src/03_randomsearch_mlflow.py")

```

Observación técnica — MLflow 3.x: con `mlflow>=2.0` un Codespace nuevo instala hoy MLflow 3.x, donde `artifact_path=` en `log_model()` está deprecado (emite un warning y sugiere `name=`). El script funciona igual; el warning es informativo y no detiene el pipeline.

**PASO
4****RandomizedSearchCV + MLflow
(Run 2)**pipeline/
03_randomsearch_mlflow.py

Muestrea 30 combinaciones de un espacio continuo (`randint` de `scipy.stats`). Más eficiente que Grid para espacios grandes. Se registra como *Run 2* en el mismo experimento, con la misma semilla y split, para que ambos runs sean comparables.

```
python - pipeline/03_randomsearch_mlflow.py

"""
Sección 9 – RandomizedSearchCV + MLflow (Run 2).

Muestrea 30 combinaciones aleatorias de un espacio continuo.
Más eficiente que GridSearch para espacios grandes.

Ejecutar: python src/03_randomsearch_mlflow.py
Prerrequisito: src/base_data.pkl (generado por 01_base_pipeline.py)
"""

import pickle
import mlflow
import mlflow.sklearn
from pathlib import Path
from scipy.stats import randint

from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, f1_score, recall_score,
    precision_score, classification_report,
)

import os
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"

# — Cargar datos —
BASE_PKL = Path("src/base_data.pkl")
if not BASE_PKL.exists():
    raise FileNotFoundError(
        "No se encontró src/base_data.pkl. "
        "Ejecuta primero: python src/01_base_pipeline.py"
    )

with open(BASE_PKL, "rb") as f:
    X_train_smote, y_train_smote, X_test, y_test = pickle.load(f)

print(f"Datos cargados – Train: {X_train_smote.shape} | Test: {X_test.shape}")

# — Configurar MLflow —
MLFLOW_DIR = Path("mlruns").resolve()
TRACKING_URI = MLFLOW_DIR.as_uri()
EXPERIMENT = "picos-intensidad-tuning"

MLFLOW_DIR.mkdir(parents=True, exist_ok=True)

mlflow.set_tracking_uri(TRACKING_URI)
mlflow.set_experiment(EXPERIMENT)
print(f"MLflow tracking URI : {TRACKING_URI}")
print(f"MLflow experiment : {EXPERIMENT}")

# — Sección 9: RandomizedSearchCV —
param_dist = {
    "max_depth": randint(2, 25),
```

```

    "min_samples_split": randint(2, 50),
    "min_samples_leaf":  randint(1, 20),
    "max_features":      ["sqrt", "log2", None],
    "criterion":         ["gini", "entropy"],
}

rs = RandomizedSearchCV(
    estimator      = DecisionTreeClassifier(random_state=123),
    param_distributions = param_dist,
    n_iter         = 30,
    cv             = StratifiedKFold(5, shuffle=True, random_state=123),
    scoring        = "f1",
    random_state   = 123,
    n_jobs         = -1,
    verbose        = 1,
)

print("\nEjecutando RandomizedSearchCV (30 combinaciones x 5 folds)...")
rs.fit(X_train_smote, y_train_smote)

print(f"\nMejores parámetros : {rs.best_params_}")
print(f"Mejor F1 CV          : {rs.best_score_: .4f}")

# — Registrar en MLflow —————
with mlflow.start_run(run_name="RandomSearch-DecisionTree") as run:
    mlflow.log_params(rs.best_params_)
    mlflow.log_param("metodo_busqueda", "RandomizedSearchCV")
    mlflow.log_param("n_iter",          30)
    mlflow.log_param("tecnica_balanceo", "SMOTE")
    mlflow.log_param("n_folds",         5)
    mlflow.log_param("random_state",    123)

    y_pred_rs = rs.best_estimator_.predict(X_test)
    mlflow.log_metrics({
        "accuracy": round(accuracy_score(y_test, y_pred_rs), 4),
        "f1":       round(f1_score(y_test, y_pred_rs), 4),
        "recall":   round(recall_score(y_test, y_pred_rs), 4),
        "precision": round(precision_score(y_test, y_pred_rs, zero_division=0), 4),
        "cv_f1_mean": round(rs.best_score_, 4),
    })

    mlflow.sklearn.log_model(
        sk_model      = rs.best_estimator_,
        artifact_path = "modelo_randomsearch",
    )
    run_id_rs = run.info.run_id
    print(f"\nRun ID RandomSearch: {run_id_rs}")

print("\n=== RESULTADO RANDOMSEARCH ===")
print(classification_report(y_test, y_pred_rs))

with open("src/run_ids.txt", "a") as f:
    f.write(f"randomsearch:{run_id_rs}\n")

print("✓ Run ID guardado en src/run_ids.txt")
print(" Siguiente paso: python src/04_comparar_runs.py")

```

**PASO
5****Comparar todos los runs**

pipeline/04_comparar_runs.py

`mlflow.search_runs()` devuelve un DataFrame ordenable por cualquier métrica. El script imprime la tabla comparativa ordenada por F1 y genera un gráfico de barras en `src/comparacion_runs.png`.

```
python - pipeline/04_comparar_runs.py
```

```
"""
Sección 10 – Comparación de runs con mlflow.search_runs().

Genera tabla comparativa ordenada por F1 y gráfico de barras.

Ejecutar: python src/04_comparar_runs.py
"""

import mlflow
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

import os
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"

# — Configurar MLflow —————
MLFLOW_DIR = Path("mlruns").resolve()
TRACKING_URI = MLFLOW_DIR.as_uri()
EXPERIMENT = "picos-intensidad-tuning"

mlflow.set_tracking_uri(TRACKING_URI)

# — Buscar todos los runs del experimento —————
exp = mlflow.get_experiment_by_name(EXPERIMENT)
if exp is None:
    raise RuntimeError(
        f"Experiment '{EXPERIMENT}' no encontrado. "
        "Ejecuta primero los scripts 02 y 03."
    )

runs = mlflow.search_runs(
    experiment_ids=[exp.experiment_id],
    order_by=["metrics.f1 DESC"],
)

print(f"Total de runs encontrados: {len(runs)}")

# — Tabla comparativa —————
cols = [
    "tags.mlflow.runName",
    "params.metodo_busqueda",
    "metrics.f1",
    "metrics.recall",
    "metrics.accuracy",
    "metrics.cv_f1_mean",
    "params.n_combinaciones",
]
cols_disp = [c for c in cols if c in runs.columns]

print("\n=== COMPARACIÓN DE RUNS (ordenado por F1 DESC) ===")
print(runs[cols_disp].to_string(index=False))

mejor = runs.iloc[0]
print(f"\n★ MEJOR RUN: {mejor.get('tags.mlflow.runName', mejor['run_id'][:8])}")
```

```

print(f" F1      : {mejor['metrics.f1']:.4f}")
print(f" Recall  : {mejor['metrics.recall']:.4f}")
print(f" Accuracy : {mejor['metrics.accuracy']:.4f}")

# — Gráfico comparativo —
metricas_plot = ["metrics.f1", "metrics.recall", "metrics.accuracy"]
titulos_plot  = ["F1-Score", "Recall", "Accuracy"]
labels = [
    str(r.get("tags.mlflow.runName", r["run_id"][:8]))
    for _, r in runs.iterrows()
]
colors = ["#2E75B6", "#C55A11", "#1E6E38", "#7030A0"]

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle(
    "Comparación de Runs – Picos Intensidad Tuning",
    fontsize=13, fontweight="bold",
)

for ax, met, titulo, col in zip(axes, metricas_plot, titulos_plot, colors):
    vals = runs[met].tolist()
    bars = ax.bar(labels, vals, color=col, alpha=0.85)
    for bar, v in zip(bars, vals):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.002,
            f"{v:.4f}", ha="center", fontsize=9, fontweight="bold",
        )
    ax.set_title(titulo, fontweight="bold")
    ax.set_ylim([max(0, min(vals) - 0.05), min(1.0, max(vals) + 0.08)])
    ax.tick_params(axis="x", rotation=20)
    ax.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
output_png = "src/comparacion_runs.png"
plt.savefig(output_png, dpi=120, bbox_inches="tight")
plt.close()

print(f"\n✓ Gráfico guardado: {output_png}")
print(" Siguiente paso: python src/05_mejor_modelo.py")

```

PASO 6

Selección programática del mejor modelo

pipeline/05_mejor_modelo.py

Toma el primer run ordenado por F1, carga el modelo directamente desde MLflow con `runs:/<id>/<artifact>` (sin buscar archivos en disco) y hace la evaluación final. Guarda `src/mejor_run.txt` para el Model Registry.

```

python - pipeline/05_mejor_modelo.py

"""
Sección 11 – Selección programática del mejor modelo.

Carga el mejor run por F1 directamente desde MLflow.

Ejecutar: python src/05_mejor_modelo.py
"""
import pickle
import mlflow

```

```

import mlflow.sklearn
from pathlib import Path
from sklearn.metrics import classification_report

import os
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"

# — Cargar datos de test —————
BASE_PKL = Path("src/base_data.pkl")
if not BASE_PKL.exists():
    raise FileNotFoundError(
        "No se encontró src/base_data.pkl. "
        "Ejecuta primero: python src/01_base_pipeline.py"
    )

with open(BASE_PKL, "rb") as f:
    _, _, X_test, y_test = pickle.load(f)

# — Configurar MLflow —————
MLFLOW_DIR = Path("mlruns").resolve()
TRACKING_URI = MLFLOW_DIR.as_uri()
EXPERIMENT = "picos-intensidad-tuning"

mlflow.set_tracking_uri(TRACKING_URI)

# — Buscar y ordenar runs por F1 —————
exp = mlflow.get_experiment_by_name(EXPERIMENT)
if exp is None:
    raise RuntimeError(
        f"Experiment '{EXPERIMENT}' no encontrado. "
        "Ejecuta primero los scripts 02 y 03."
    )

runs = mlflow.search_runs(
    experiment_ids=[exp.experiment_id],
    order_by=["metrics.f1 DESC"],
)

# — Identificar el mejor run —————
mejor_run = runs.iloc[0]
mejor_id = mejor_run["run_id"]
mejor_nombre = mejor_run.get("tags.mlflow.runName", "desconocido")

print("=" * 50)
print("SECCIÓN 11 – Selección del mejor modelo")
print("=" * 50)
print(f"Mejor run : {mejor_nombre}")
print(f"Run ID : {mejor_id}")
print(f"F1 : {mejor_run['metrics.f1']:.4f}")
print(f"Recall : {mejor_run['metrics.recall']:.4f}")
print(f"Accuracy : {mejor_run['metrics.accuracy']:.4f}")
print(f"CV F1 mean : {mejor_run.get('metrics.cv_f1_mean', 'N/A')}")

# — Cargar el modelo directamente desde MLflow —————
artifact = "modelo_gridsearch" if "Grid" in mejor_nombre else "modelo_randomsearch"
model_uri = f"runs://{mejor_id}/{artifact}"

print(f"\nCargando modelo desde: {model_uri}")
modelo_produccion = mlflow.sklearn.load_model(model_uri)

print(f"Tipo de modelo: {type(modelo_produccion).__name__}")
print(f"Parámetros : {modelo_produccion.get_params()}")

# — Evaluación final completa —————
y_pred_final = modelo_produccion.predict(X_test)
print("\n=== EVALUACIÓN FINAL DEL MEJOR MODELO ===")

```

```
print(classification_report(y_test, y_pred_final))

# — Guardar info para el Model Registry —————
with open("src/mejor_run.txt", "w") as f:
    f.write(f"{mejor_id}\n{mejor_nombre}\n{artifact}")

print("✓ Info guardada en src/mejor_run.txt")
print("  Siguiente paso: python src/06_model_registry.py")
```

PASO 7

MLflow Model Registry (Staging → Production)

pipeline/
06_model_registry.py

Registra el mejor modelo en el Model Registry y lo promueve a **Staging**. El flujo completo es: Run → Registered Model → Staging → (Production). La promoción a Production queda comentada hasta validar en Staging.

```
python - pipeline/06_model_registry.py

"""
Sección 12 – Registro del mejor modelo en MLflow Model Registry.

Flujo: Run → Registered Model → Staging → (Production)

Ejecutar: python src/06_model_registry.py
Prerrequisito: src/mejor_run.txt (generado por 05_mejor_modelo.py)
"""

import mlflow
import mlflow.sklearn
from mlflow import MlflowClient
from pathlib import Path

import os
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"

# — Configurar MLflow —————
MLFLOW_DIR = Path("mlruns").resolve()
TRACKING_URI = MLFLOW_DIR.as_uri()
MODEL_REGISTRY = "PicosIntensidad-Clasificador"

mlflow.set_tracking_uri(TRACKING_URI)

# — Leer el mejor run —————
MEJOR_RUN_FILE = Path("src/mejor_run.txt")
if not MEJOR_RUN_FILE.exists():
    raise FileNotFoundError(
        "No se encontró src/mejor_run.txt. "
        "Ejecuta primero: python src/05_mejor_modelo.py"
    )

with open(MEJOR_RUN_FILE) as f:
    lines = f.read().strip().split("\n")
    mejor_id = lines[0]
    mejor_nom = lines[1]
    artifact = lines[2]

print("=" * 50)
print("SECCIÓN 12 – MLflow Model Registry")
print("=" * 50)
print(f"Run a registrar : {mejor_nom}")
```

```

print(f"Run ID      : {mejor_id[:12]}...")
print(f"Artifact path : {artifact}")
print(f"Registry name  : {MODEL_REGISTRY}")

# — Registrar el mejor modelo —————
model_uri = f"runs://{mejor_id}/{artifact}"
result    = mlflow.register_model(model_uri, MODEL_REGISTRY)
version   = result.version
print(f"\n✓ Modelo registrado: {MODEL_REGISTRY} v{version}")

# — Promover a Staging —————
client = MlflowClient(tracking_uri=TRACKING_URI)
client.transition_model_version_stage(
    name     = MODEL_REGISTRY,
    version  = version,
    stage    = "Staging",
)
print(f"\n✓ Modelo en Staging: {MODEL_REGISTRY} v{version}")

# — Verificar estado —————
model_details = client.get_model_version(MODEL_REGISTRY, version)
print(f"\nEstado del modelo registrado:")
print(f" Nombre   : {model_details.name}")
print(f" Versión  : {model_details.version}")
print(f" Stage    : {model_details.current_stage}")
print(f" Run ID   : {model_details.run_id[:12]}...")

# — Verificar carga desde el Registry —————
print(f"\nVerificando carga desde Staging...")
modelo_staging = mlflow.sklearn.load_model(f"models://{MODEL_REGISTRY}/Staging")
print(f"\n✓ Modelo cargado desde Staging: {type(modelo_staging).__name__}")

# — Opcional: promover a Production —————
# Descomentar cuando el modelo haya pasado validación en Staging:
#
# client.transition_model_version_stage(
#     name=MODEL_REGISTRY, version=version, stage="Production"
# )
# print(f" Modelo en Production: {MODEL_REGISTRY} v{version}")

print("\n" + "=" * 50)
print("TEMA 6 COMPLETADO")
print("=" * 50)
print(f" Modelo registrado : {MODEL_REGISTRY} v{version} (Staging)")
print(f" MLflow UI         : mlflow ui --host 0.0.0.0 --port 5000")
print(f" Gráfico           : src/comparacion_runs.png")

```

Observación técnica — `transition_model_version_stage` deprecado: en MLflow 3.x los *stages* del Model Registry (Staging/Production) están deprecados desde la versión 2.9 y se eliminarán en el futuro; el método emite un `FutureWarning` pero todavía funciona. Al registrar, MLflow 3.x también avisa que toma el modelo desde `models:/...` en lugar del artefacto del run — se resuelve solo. Para código a prueba de futuro, la alternativa es usar *alias* y *tags* del modelo en vez de stages.

PASO 8

Ejecutar todo el tema en secuencia

pipeline/run_pipeline.py

El orquestador ejecuta los 6 scripts en orden y se detiene si alguno falla. Su lista interna usa las rutas correctas `pipeline/...`.

```
python - pipeline/run_pipeline.py

"""
run_tema6.py - Orquestador: ejecuta las 6 secciones del Tema 6 en secuencia.

Uso desde la terminal del Codespace:
python tema6/run_tema6.py

Equivale a ejecutar uno a uno:
python tema6/01_base_pipeline.py
python tema6/02_gridsearch_mlflow.py
python tema6/03_randomsearch_mlflow.py
python tema6/04_comparar_runs.py
python tema6/05_mejor_modelo.py
python tema6/06_model_registry.py
"""

import subprocess
import sys
import time
import logging
from pathlib import Path

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | TEMA6 | %(levelname)s | %(message)s",
    datefmt="%H:%M:%S",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler("tema6_run.log"),
    ],
)

log = logging.getLogger("tema6")

SCRIPTS = [
    ("Secciones 1-7 : Base pipeline", "pipeline/01_base_pipeline.py"),
    ("Sección 8 : GridSearchCV + MLflow", "pipeline/02_gridsearch_mlflow.py"),
    ("Sección 9 : RandomizedSearchCV + MLflow", "pipeline/03_randomsearch_mlflow.py"),
    ("Sección 10 : Comparar runs", "pipeline/04_comparar_runs.py"),
    ("Sección 11 : Mejor modelo", "pipeline/05_mejor_modelo.py"),
    ("Sección 12 : Model Registry", "pipeline/06_model_registry.py"),
]

def ejecutar(nombre: str, script: str) -> bool:
    """Ejecuta un script y retorna True si tuvo éxito."""
    inicio = time.time()
    log.info(">>> %s", nombre)
    resultado = subprocess.run(
        [sys.executable, script],
        capture_output=False, # muestra output en tiempo real
    )
    duracion = round(time.time() - inicio, 2)
    if resultado.returncode == 0:
        log.info("<<< OK: %s (%.2f s)", nombre, duracion)
        return True
    else:
        log.error("XXX FALLO: %s (%.2f s)", nombre, duracion)
        return False

log.info("=" * 55)
log.info(" TEMA 6 - Ajuste y Seguimiento de Modelos")
log.info("=" * 55)

resumen = []
```



```

for nombre, script in SCRIPTS:
    ok = ejecutar(nombre, script)
    resumen.append((nombre, ok))
    if not ok:
        log.error("Pipeline detenido. Corrige el error y reintenta.")
        log.error("Puedes ejecutar el script individualmente para depurar:")
        log.error("  python %s", script)
        sys.exit(1)

log.info("=" * 55)
log.info(" TEMA 6 COMPLETADO")
log.info("=" * 55)
for nombre, ok in resumen:
    estado = "OK" if ok else "XX"
    log.info(" [%s] %s", estado, nombre)

log.info("")
log.info(" Resultados:")
log.info("   Gráfico comparativo : tema6/comparacion_runs.png")
log.info("   Modelo registrado   : MLflow Model Registry (Staging)")
log.info("   Log de ejecución    : tema6_run.log")
log.info("")
log.info(" Para ver MLflow UI en Codespace:")
log.info("   mlflow ui --host 0.0.0.0 --port 5000")
log.info("   Luego: pestaña PORTS -> clic en globo del puerto 5000")
# MLFLOW_ALLOW_FILE_STORE=true mlflow ui --host 0.0.0.0 --port 5000

```

Observación técnica — comandos individuales: los *docstrings* y los mensajes «Siguiente paso» de cada script dicen `python src/0N_...py`, pero los archivos están en `pipeline/`. El comando correcto para ejecutarlos uno a uno es `python pipeline/0N_...py`. El orquestador `run_pipeline.py` sí usa las rutas correctas, así que el pipeline completo funciona sin cambios.

bash — Terminal del Codespace — flujo completo

```

cd /workspaces/tu-repo

# Opción A: todo en secuencia (recomendado)
python pipeline/run_pipeline.py

# Opción B: script por script (para depurar)
python pipeline/01_base_pipeline.py
python pipeline/02_gridsearch_mlflow.py
python pipeline/03_randomsearch_mlflow.py
python pipeline/04_comparar_runs.py
python pipeline/05_mejor_modelo.py
python pipeline/06_model_registry.py

```

bash — Ver la MLflow UI (segunda terminal)

```

# En una SEGUNDA terminal (Ctrl+` → botón +):
mlflow ui --host 0.0.0.0 --port 5000
# Usar 0.0.0.0 para que Codespaces reenvíe el puerto.
# Luego: pestaña PORTS → clic en el globo del puerto 5000

```

✓ **Resultado esperado:** dos runs (`GridSearch-DecisionTree` y `RandomSearch-DecisionTree`) en el experimento `picos-intensidad-tuning`, un gráfico en `src/comparacion_runs.png` y el modelo `PicosIntensidad-Clasificador` registrado en **Staging**.

6. Referencia rápida de comandos

Comando	Para qué sirve
<code>python pipeline/run_pipeline.py</code>	Ejecuta las 6 secciones del tema en secuencia
<code>python pipeline/01_base_pipeline.py</code>	Genera <code>src/base_data.pkl</code> (split reutilizable)
<code>python pipeline/02_gridsearch_mlflow.py</code>	GridSearchCV + registro del Run 1 en MLflow
<code>python pipeline/03_randomsearch_mlflow.py</code>	RandomizedSearchCV + registro del Run 2
<code>python pipeline/04_comparar_runs.py</code>	Tabla comparativa + gráfico de barras
<code>python pipeline/05_mejor_modelo.py</code>	Selecciona y carga el mejor run por F1
<code>python pipeline/06_model_registry.py</code>	Registra el ganador y lo promueve a Staging
<code>mlflow ui --host 0.0.0.0 --port 5000</code>	Lanza la UI de MLflow (requerido en Codespaces)

7. ¿Qué aprendiste en este tema?

Hyperparameter Tuning

Buscaste los mejores hiperparámetros con GridSearchCV y RandomizedSearchCV. El tuning siempre parte de una línea base y usa la misma semilla, split y métrica para ser comparable.

GridSearch vs. RandomSearch

Grid evalúa TODAS las combinaciones (garantiza el óptimo en el espacio dado). Random muestrea `n_iter` combinaciones de un espacio continuo — más eficiente para espacios grandes.

Integración tuning + MLflow

Cada método genera un Run con sus parámetros, métricas y modelo como artefacto. La clave: misma semilla, mismo split y misma métrica en todos los runs.

Comparación programática

`mlflow.search_runs()` retorna un DataFrame ordenado por cualquier métrica y permite cargar el mejor modelo directamente desde MLflow, sin buscar archivos en disco.

MLflow Model Registry

El ganador se registra con `mlflow.register_model()` y se promueve a Staging. El flujo es: Run → Registered Model → Staging → Production.

■ **En el Tema 7...** automatizaremos este pipeline con **CI/CD**: un workflow de GitHub Actions que, en cada `git push` a `main`, ejecuta linting, tests, re-entrenamiento, validación de métricas (quality gate) y build de la imagen Docker — sin intervención manual.

8. Checklist final

- ☐ Instalé las dependencias con `pip install -r requirements.txt`.

- ☐ Ejecuté `python pipeline/01_base_pipeline.py` y se creó `src/base_data.pkl`.
- ☐ Corrí GridSearch y RandomSearch; veo dos runs en el experimento `picos-intensidad-tuning`.
- ☐ Comparé los runs y se generó `src/comparacion_runs.png`.
- ☐ Seleccioné el mejor modelo por F1 y lo registré en el Model Registry (Staging).
- ☐ Abrí la MLflow UI en el puerto 5000 reenviado por la pestaña PORTS.